

Lab Exercise: From SGD to AdamW

Building an Interactive Visual Tool to See How Optimizers Learn

1. Objective

This lab has two intertwined goals. First, you will implement seven optimization algorithms — SGD, SGD with Momentum, Nesterov Accelerated Gradient (NAG), AdaGrad, RMSProp, Adam, and AdamW — entirely from scratch using NumPy. Second, and central to this version of the exercise, you will package your implementations inside an interactive visual software application rather than a static notebook of plots.

Instead of generating a fixed set of pre-computed figures, your software should let a user choose a loss surface, pick one or more optimizers, tune their hyperparameters with sliders, and watch the optimization trajectory animate step-by-step across the surface in real time. The aim is to build a small, reusable “optimizer playground” that could be used to teach the next batch of students how these algorithms actually behave.

Built-in optimizer implementations such as `torch.optim.SGD`, `torch.optim.Adam`, `tf.keras.optimizers.Adam`, or `keras.optimizers` must NOT be used anywhere. You must implement every update rule yourself. Libraries for numerics (NumPy), plotting/animation (Matplotlib, Plotly), and UI (ipywidgets, Streamlit, Dash, Tkinter, PyQt, or a browser front end) are all permitted — see Section 3.

2. Learning Outcomes

- Derive and implement the update rule of each optimizer from first principles.
- Explain why curvature (conditioning) of a loss surface causes zig-zagging, and how momentum, adaptive scaling, and decoupled weight decay each respond to it.
- Translate a mathematical algorithm into a stateful, interactive software component (not just a one-shot script).
- Design a simple, usable interface: input controls, live feedback, and clear visual encoding (color, animation, legends).
- Use the finished tool to reason about optimizer behaviour empirically, by playing with it, rather than only by reading a plot someone else generated.

3. What You Are Building: The Optimizer Visualizer

Your deliverable is a single interactive application (desktop, notebook-widget, or browser-based) with two panels/modes described in Parts A and B below. Choose ONE of the following tech stacks — all are acceptable, pick whichever you are comfortable with:

Stack	How the UI is built	How animation works
Python + Matplotlib + ipywidgets	Sliders/dropdowns via ipywidgets inside a Jupyter notebook	matplotlib.animation.FuncAnimation or manual redraw on slider change
Python + Streamlit	st.slider, st.selectbox, st.multiselect in a .py app run with 'streamlit run'	st.pyplot / st.plotly_chart redrawn on a loop or via a “Play” button + session state
Python + Plotly Dash	dcc.Slider, dcc.Dropdown, dcc.Interval components	dcc.Interval callback updates the figure every N milliseconds

Stack	How the UI is built	How animation works
Python + Tkinter / PyQt	Native GUI widgets (Scale, ComboBox, Button)	Timer/after() callback redraws an embedded Matplotlib canvas
HTML/CSS/JavaScript	Native form controls (range inputs, checkboxes) + Canvas or D3.js/Chart.js	requestAnimationFrame or setInterval driving canvas redraws

Regardless of stack, the application must expose the controls and views listed in Section 4 (Part A) and Section 6 (Part B).

PART A — The Optimizer Playground on a 2D Loss Surface

A0. Background: the elongated bowl

Consider the loss function $L(x, y) = x^2 + 50y^2$, with gradient $\nabla L(x, y) = [2x, 100y]^T$ and global minimum at $(0, 0)$. Because curvature along y is $50\times$ larger than along x , the loss surface is a narrow, elongated bowl — the classic test case for exposing zig-zagging behaviour. All trajectories should start at $(x_0, y_0) = (8, 8)$.

Implement each optimizer as a small, self-contained Python class or function with a `.step(gradient)` method that returns the updated parameters and retains any internal state (velocity, moving averages, timestep) between calls — this statefulness is what makes it possible to drive the optimizer one animation frame at a time instead of only in a single batch loop.

A1. Implement the seven update rules

1. SGD: $\theta_{t+1} = \theta_t - \eta g_t$
2. SGD with Momentum ($\beta = 0.9$): $v_t = \beta v_{t-1} + (1-\beta)g_t$, $\theta_{t+1} = \theta_t - \eta v_t$
3. NAG: evaluate the gradient at a look-ahead point $\theta_t - \beta v_{t-1}$ rather than at θ_t itself, then apply the momentum update.
4. AdaGrad: $G_t = G_{t-1} + g_t^2$, $\theta_{t+1} = \theta_t - \eta g_t / \sqrt{G_t + \epsilon}$
5. RMSProp ($\beta = 0.9$): $v_t = \beta v_{t-1} + (1-\beta)g_t^2$, $\theta_{t+1} = \theta_t - \eta g_t / \sqrt{v_t + \epsilon}$
6. Adam ($\beta_1=0.9, \beta_2=0.999$): compute m_t, v_t , bias-correct to $\hat{m}_t, \hat{v}_t$, then $\theta_{t+1} = \theta_t - \eta \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$
7. AdamW ($\lambda \approx 10^{-3}$): identical to Adam but with decoupled weight decay: $\theta_{t+1} = \theta_t - \eta (\hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon) + \lambda \theta_t)$

Use a maximum of 500 iterations and a default learning rate of 0.01; document any other hyperparameter defaults you choose inside the app itself (e.g. in a settings/info panel), not just in a report.

A2. Build the interactive controls

Your app's Part-A panel must provide the following live controls, all of which trigger an immediate visual update (no re-running a script by hand):

- A dropdown or checkbox group to choose the loss surface: $L_1 = x^2+10y^2$, $L_2 = x^2+50y^2$ (default), $L_3 = x^2+100y^2$, $L_4 = x^2+1000y^2$ (see A5).
- A multi-select list of optimizers so the user can overlay any subset of the seven trajectories at once, each drawn in a distinct colour with a legend.
- A slider for the learning rate η (suggested range 0.0001–0.5, log scale if your UI supports it).
- Sliders for β (momentum/RMSProp), β_1, β_2 (Adam/AdamW), and λ (AdamW weight decay).
- A numeric field or slider for the starting point (x_0, y_0) , defaulting to (8, 8).
- Play / Pause / Step / Reset controls that animate the trajectory point-by-point rather than drawing it instantly, plus a speed control.

A3. Build the two synchronized views

- View 1 — Contour map: a filled contour plot of the selected loss surface with the global minimum marked (★ at the origin), and the live trajectory drawn as a connected, growing path with a marker at the current point for every selected optimizer.
- View 2 — Loss curve: $L(\theta_t)$ vs. iteration t , updating in lock-step with View 1 as the animation plays, using the same colour per optimizer as View 1.

Keep the two views on screen simultaneously (side by side or stacked) so a user can watch the path in parameter space and the corresponding loss decrease at the same time.

A4. Explain-as-you-go panel

Add a small text/info panel in the app (can be a sidebar, tooltip, or expandable section) that updates or can be toggled per optimizer, briefly explaining in your own words:

- Why NAG's look-ahead gradient can reduce overshoot compared to plain Momentum.
- Why AdaGrad gives different effective learning rates to different parameters.
- How RMSProp fixes AdaGrad's continuously-shrinking learning rate.
- The role of decoupled weight decay in AdamW versus L2 regularization folded into the gradient.

A5. Conditioning explorer

Wire the loss-surface dropdown from A2 to actually re-run every optimizer live on L_1 , L_2 , L_3 , and L_4 (increasing condition number). Let the user flip between them and observe, without leaving the app, how the same hyperparameters behave very differently as the bowl becomes more elongated. Add one or two sentences (in your report) connecting the condition number of the Hessian to how narrow the bowl looks and how badly SGD zig-zags.

A6. Learning-rate sensitivity explorer

Because η is already a live slider from A2, use the app itself to sweep $\eta \in \{0.001, 0.01, 0.1\}$ for each optimizer and observe convergence, oscillation, or divergence directly on screen. Capture a screenshot or short screen-recording of at least one case where an optimizer diverges and one where it converges smoothly, and include both in your submission.

A7. Reflection questions — Part A

1. Which optimizer shows the strongest zig-zag on the default bowl, and why?
2. Which optimizer(s) most visibly reduce oscillation, and through what mechanism?
3. Which optimizer moves most efficiently along the shallow (x) direction while the y direction is corrected quickly?
4. Which optimizers use parameter-wise adaptive learning rates, and how can you tell just from watching the animation?
5. What visual difference do you observe between AdaGrad and RMSProp as the animation runs past ~200 iterations?
6. What visual difference do you observe between RMSProp and Adam?
7. On this simple 2D problem, does AdamW visibly differ from Adam? Why or why not, given λ is small?
8. As you increase the condition number from L_1 to L_4 using your app, which optimizers remain stable and which start to oscillate or diverge at $\eta = 0.01$?

PART B — Extending the Tool to a Real Neural Network

Extend the same application (a second tab/panel, or a separate mode) so it can train a small Multi-Layer Perceptron on a real dataset — e.g. the Breast Cancer Wisconsin dataset — using whichever optimizer(s) the user selects, and visualize training live rather than only after the fact.

B1. Network and data

- Architecture: Input $\rightarrow$ Dense(16) $\rightarrow$ ReLU $\rightarrow$ Dense(8) $\rightarrow$ ReLU $\rightarrow$ Dense(1) $\rightarrow$ Sigmoid.
- Implement forward propagation, binary cross-entropy loss, and full backpropagation by hand (NumPy only).
- Load the dataset, split into train/test, standardize features, and report sample counts and feature counts — display these numbers in the app's UI, not just printed to a console.
- Reuse the exact same seven optimizer classes/functions from Part A — do not write a second implementation.

B2. Live training dashboard

Add controls to select one or more optimizers, set the learning rate and other hyperparameters, set epochs/batch size, and press “Train”. While training runs, the dashboard should update, ideally every epoch (not just at the end):

- Training loss vs. epoch (one line per selected optimizer, shared colour scheme with Part A).
- Validation/test loss vs. epoch.
- Accuracy vs. epoch.
- A live “effective learning rate” readout for one representative weight, for AdaGrad, RMSProp, Adam, and AdamW — i.e. $\eta/\sqrt{(G_t+\epsilon)}$ or $\eta/\sqrt{(\hat{v}_t+\epsilon)}$ plotted over epochs, so the user can see it shrink or stabilize as training proceeds.

B3. Comparison table

Once training finishes for all selected optimizers, have the app render (in the UI, e.g. as an HTML/Streamlit/Dash table, or an embedded Matplotlib table) the following summary, auto-computed from the run — do not fill this in by hand:

Optimizer	Final Train Loss	Final Test Loss	Train Acc.	Test Acc.	Convergence Epoch
SGD					
Momentum					
NAG					
AdaGrad					
RMSProp					
Adam					
AdamW					

(“Convergence epoch”: the first epoch at which validation loss reaches within 1% of its final value — compute this automatically in code.)

B4. Reflection questions — Part B

1. Why does plain SGD tend to zig-zag on an elongated/ill-conditioned surface, and did you see this echoed in the neural-network training curves?
2. How does Momentum reduce oscillation, mechanically?
3. How is NAG different from Momentum, and did the live app make that difference visible or was it subtle?

4. Why does AdaGrad reduce the learning rate for parameters with consistently large gradients?
5. Why can AdaGrad eventually become too slow? Did your effective-learning-rate plot (B2) show this?
6. How does RMSProp solve AdaGrad's main weakness?
7. What are the roles of m_t and v_t in Adam?
8. Why is bias correction required early in Adam's training?
9. How does Adam combine ideas from Momentum and RMSProp?
10. What is the purpose of decoupled weight decay in AdamW, and how does it differ from adding $\lambda\theta$ directly to the gradient?
11. Which optimizer converged fastest in your dashboard?
12. Which optimizer produced the best test performance?
13. Is the fastest-converging optimizer necessarily the one with the best generalization? What did your table show?
14. Which optimizer was most sensitive to the learning-rate slider?
15. What happens, both in Part A and Part B, as the condition number of the problem increases?
16. Having built and played with the tool yourself, which optimizer would you pick for a new deep-learning project, and why?

4. Software Design Requirements

Beyond correctness of the math, part of your grade covers whether this is usable software, not a one-off script:

- Clear separation between the optimizer logic (pure functions/classes, no plotting code inside them) and the UI/animation layer.
- Sensible default values pre-loaded so the app is useful the moment it opens, with no crash on default settings.
- Input validation: the app should not crash if a user enters an invalid learning rate, e.g. 0 or negative.
- A visible legend, axis labels, and a title on every plot; consistent colour-per-optimizer across every view in the app.
- A short in-app “How to use this tool” note or README.

5. Submission Requirements

1. Full source code of the application (single script, notebook, or small project folder with a clear entry point and run instructions).
2. From-scratch implementations of all seven optimizers, used identically in Part A and Part B.
3. The neural-network implementation (forward pass, loss, backprop) with no autograd or built-in optimizer.
4. A short screen-recording (2–4 minutes) or a sequence of annotated screenshots demonstrating the app: changing the optimizer selection, tuning a slider live, and watching an animation play.
5. The auto-generated comparison table from B3.
6. Written answers to all reflection questions in Sections A7 and B4.
7. A one-page conclusion summarizing what you learned across $\text{SGD} \rightarrow \text{Momentum} \rightarrow \text{NAG} \rightarrow \text{AdaGrad} \rightarrow \text{RMSProp} \rightarrow \text{Adam} \rightarrow \text{AdamW}$, and one paragraph on what you would improve about your own tool with more time.

6. Restrictions

Do not import or call:

- `torch.optim` (any optimizer class)
- `tensorflow.keras.optimizers` / `keras.optimizers`
- Any other pre-built optimizer, scheduler, or auto-differentiation engine for the gradient/update-rule computation.

NumPy, Pandas, Matplotlib/Plotly, standard dataset utilities (e.g. `sklearn.datasets`, `sklearn.model_selection` for splitting), and any UI/animation library listed in Section 3 are all permitted. The objective is to understand, implement, and interactively demonstrate the optimization algorithms — not to call an existing library implementation.

7. Evaluation Rubric

Component	Weight
Correctness of all 7 optimizer implementations (Part A math)	25%
Correctness of NN forward/backprop and training loop (Part B)	20%
Interactivity & usability of the visual tool (controls, animation, live views)	25%
Quality/clarity of comparison table, plots, and app polish	10%
Depth of reflection-question answers	15%
Conclusion & code readability	5%